



Mata Kuliah : PCVK
Program Studi : D4 – Teknik Informatika
Semester : 5

Kelas : TI – 3C
NIM : 244107020136
Nama : RAKAGALI RESDA KRISANDI PUTRA
Jobsheet Ke- : MINGGU Ke 2
Github : https://github.com/Rageel-28/PCVK_Ganjil_2026
Collab : <https://colab.research.google.com/drive/1bZ4AOqdfNB5G8FQTScDBgE1Ez79bCyvX?usp=sharing>

TUGAS 1

- Persiapan: Import Library dan Gambar**
Langkah pertama adalah menghubungkan Google Drive atau mengunggah gambar secara langsung, lalu mengimpor pustaka yang dibutuhkan.

```
# Import library yang dibutuhkan
import cv2 as cv
from google.colab.patches import cv2_imshow
from skimage import io
import matplotlib.pyplot as plt
import numpy as np

# Cara 1: Menggunakan file upload langsung dari komputer (Pilih salah satu cara)
from google.colab import files
uploaded = files.upload()
```

... ktp.jpeg
ktp.jpeg(image/jpeg) - 255993 bytes, last modified: n/a - 100% done
Saving ktp.jpeg to ktp.jpeg



```
img = cv.imread('ktp.jpeg')
cv2.imshow(img)

print("Resolusi Gambar : ", img.shape[1], "x", img.shape[0])
```



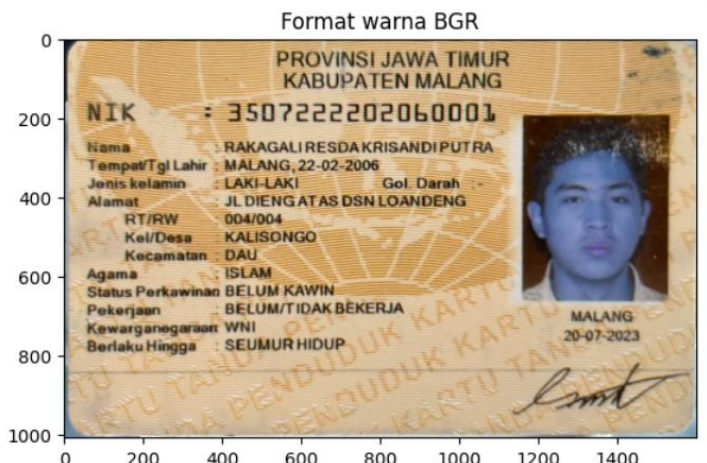
Resolusi Gambar : 1600 x 1008

2 Menampilkan Gambar

```
img = cv.imread( ktp.jpeg )

# Menampilkan gambar dengan channel warna bawaan OpenCV (BGR)
plt.imshow(img)
plt.title("Format warna BGR")
plt.show()

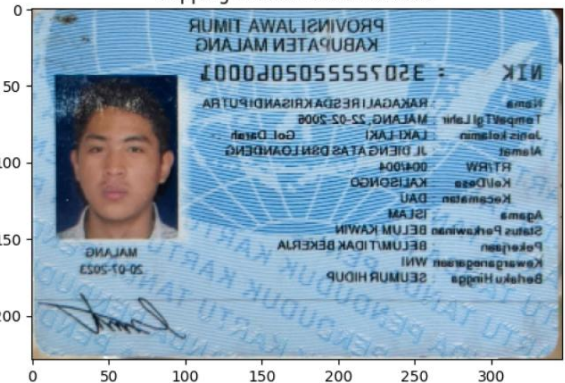
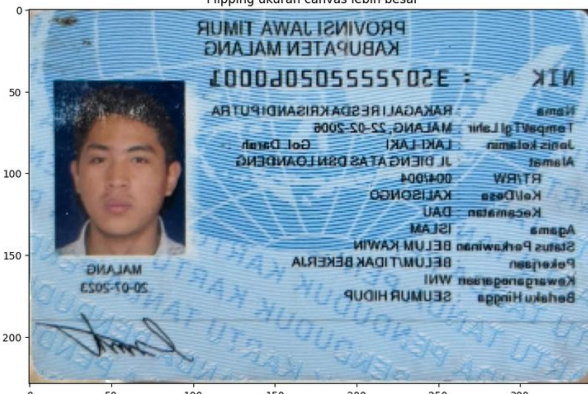
# Konversi channel BGR menjadi RGB
img2 = cv.cvtColor(img, cv.COLOR_BGR2RGB)
plt.imshow(img2)
plt.title("Format warna RGB")
plt.show()
```





	Format warna RGB
3	Menampilkan citra Grayscale: <pre>3. Menampilkan citra Grayscale: img_gray = cv.imread('ktp.jpeg', cv.IMREAD_GRAYSCALE) # Tanpa cmap plt.subplot(1, 2, 1) plt.imshow(img_gray) plt.title('Tanpa cmap') # Dengan cmap 'gray' plt.subplot(1, 2, 2) plt.imshow(img_gray, cmap='gray') plt.title('Dengan cmap') plt.show() # Dengan cmap 'magma' plt.figure() plt.imshow(img_gray, cmap='magma') plt.title('Dengan cmap magma') plt.show()</pre>
4	Melakukan resizing <pre># Konversi BGR ke RGB terlebih dahulu imgRGB = cv.cvtColor(img, cv.COLOR_BGR2RGB) # Mengubah ukuran menggunakan resize img_output = cv.resize(imgRGB, (347, 229)) # Menampilkan hasil resize plt.imshow(img_output) plt.title("Hasil Resizing (347x229)") plt.show()</pre>
5	Melakukan Flipping



<pre># Melakukan flipping horizontal (nilai 1) imgFlip = cv.flip(img_output, 1) # Menampilkan dengan ukuran kanvas default plt.imshow(imgFlip) plt.title("Flipping ukuran canvas default") plt.show() # Menampilkan ukuran canvas yang lebih besar fig = plt.figure(figsize=(10, 10)) ax = fig.add_subplot(1, 1, 1) ax.imshow(imgFlip) ax.set_title("Flipping ukuran canvas lebih besar") plt.show()</pre>	Flipping ukuran canvas default  Flipping ukuran canvas lebih besar 
6	Membuat bentuk Geometri 2D dari OpenCV. Diawali dengan pembuatan black image dengan tipe data int16.



```
# Membuat kanvas hitam (tipe data int16)
black_img = np.zeros(shape=(530, 530, 3), dtype=np.int16)

# Menambahkan persegi panjang
cv.rectangle(black_img, pt1=(290, 0), pt2=(520, 130), color=(0, 0, 255), thickness=1)
cv.rectangle(black_img, pt1=(250, 250), pt2=(350, 350), color=(255, 0, 0), thickness=1)

# Menambahkan lingkaran
cv.circle(black_img, center=(150, 150), radius=60, color=(0, 255, 0), thickness=8)
cv.circle(black_img, center=(400, 400), radius=50, color=(0, 255, 0), thickness=-1)

# Menambahkan garis
cv.line(black_img, pt1=(0, 0), pt2=(530, 530), color=(255, 255, 255), thickness=5)

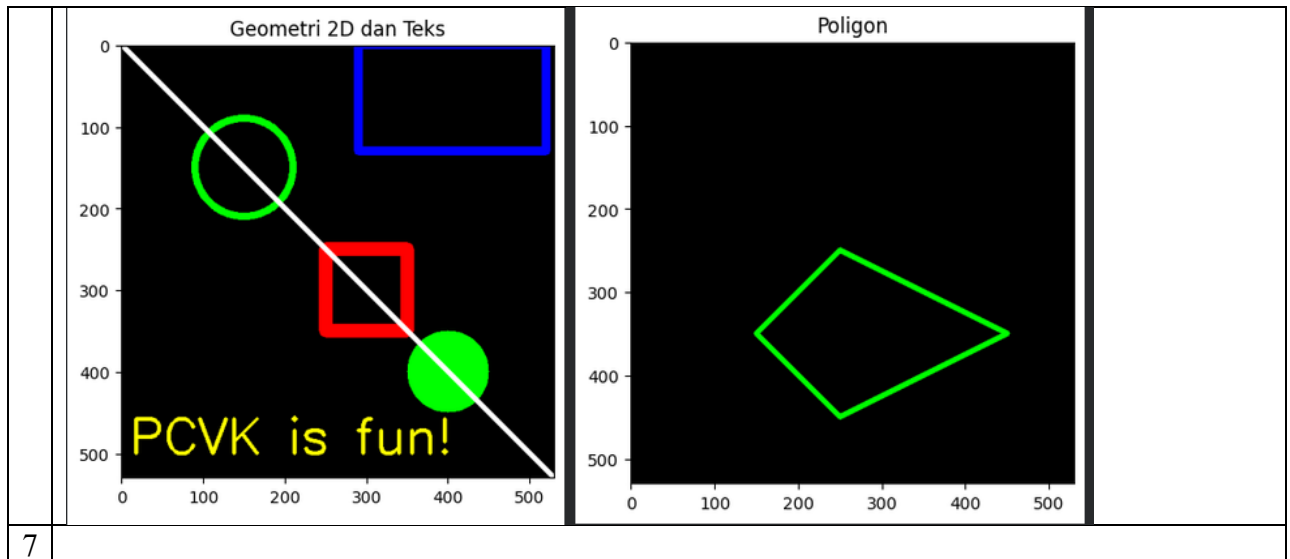
# Menambahkan teks
font = cv.FONT_HERSHEY_SIMPLEX
cv.putText(black_img, text='PCVK is fun!', org=(10, 500), fontFace=font, fontScale=2,
           color=(255, 255, 0), thickness=3, lineType=cv.LINE_AA)

plt.imshow(black_img)
plt.title("Geometri 2D dan Teks")
plt.show()

# Membuat poligon (tipe data int32)
black_img2 = np.zeros(shape=(530, 530, 3), dtype=np.int32)
vertices = np.array([[150, 350], [250, 250], [450, 350], [250, 450]], dtype=np.int32)
pts = vertices.reshape((-1, 1, 2))

cv.polylines(black_img2, [pts], isClosed=True, color=(0, 255, 0), thickness=5)

plt.imshow(black_img2)
plt.title("Poligon")
plt.show()
```



PERTANYAAN PRAKTIKUM

- 1 Tampilkan satu citra bebas menggunakan `cv2.imshow()` dan `plt.imshow()` tanpa konversi warna apa pun. Mengapa hasilnya berbeda? Buktikan dengan mencetak nilai satu piksel yang sama dari kedua cara pembacaan tersebut.

```
import cv2 as cv
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow
from skimage import io
import numpy as np
import urllib.request

# URL gambar uji standar dari OpenCV
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg'

# Membaca sebagai RGB menggunakan skimage
img_sk = io.imread(url)

# Membaca sebagai BGR menggunakan OpenCV dari URL
req = urllib.request.urlopen(url)
arr = np.asarray(bytearray(req.read()), dtype=np.uint8)
img_cv = cv.imdecode(arr, cv.IMREAD_COLOR)

print("1. Tampilan cv2_imshow (Asumsi BGR - Warna Normal):")
cv2_imshow(img_cv)

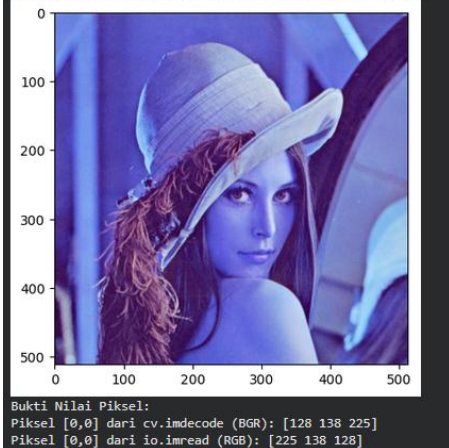
print("\n2. Tampilan plt.imshow (Asumsi RGB - Warna Tertukar/Biru):")
plt.imshow(img_cv)
plt.show()

print("Bukti Nilai Piksel:")
print("Piksel [0,0] dari cv.imdecode (BGR):", img_cv[0,0])
print("Piksel [0,0] dari io.imread (RGB):", img_sk[0,0])
```





2. Tampilan plt.imshow (Asumsi RGB - Warna Tertukar/Biru):



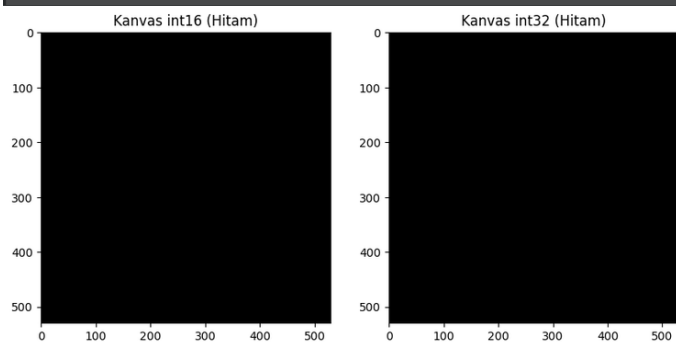
Hasilnya berbeda karena kedua fungsi tersebut mengasumsikan format urutan warna (channel) yang berbeda. Fungsi `cv2.imshow()` menganggap matriks gambar masukannya berformat BGR (Blue, Green, Red) sesuai standar OpenCV. Sebaliknya, `plt.imshow()` dari pustaka Matplotlib mengasumsikan gambar masukannya sudah berurutan RGB (Red, Green, Blue). Karena gambar dibaca dengan `cv2.imread()` yang menghasilkan BGR, jika langsung ditampilkan dengan `plt.imshow()`, warna merah dan biru pada gambar akan terlihat tertukar (misalnya wajah menjadi kebiruan)

2. Buat kanvas hitam berukuran sama dengan tipe data `int16` dan `int32`. Bandingkan `dtype`, `min()`, `max()`, dan `nbytes` keduanya. Apakah tampilannya berbeda? Apakah ukuran memorinya berbeda? Jelaskan alasannya.

```
img_int16 = np.zeros((530, 530, 3), dtype=np.int16)
img_int32 = np.zeros((530, 530, 3), dtype=np.int32)

# Menampilkan kedua kanvas berdampingan
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5))
ax1.imshow(img_int16)
ax1.set_title("Kanvas int16 (Hitam)")
ax2.imshow(img_int32)
ax2.set_title("Kanvas int32 (Hitam)")
plt.show()

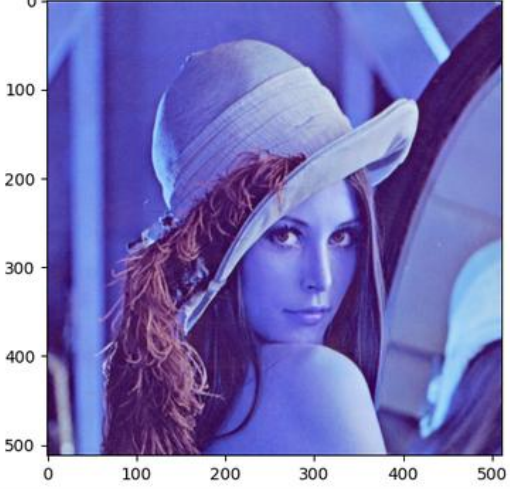
print("INT16 -> dtype:", img_int16.dtype, "| min:", img_int16.min(), "| max:", np.iinfo(img_int16.dtype).max, "
print("INT32 -> dtype:", img_int32.dtype, "| min:", img_int32.min(), "| max:", np.iinfo(img_int32.dtype).max, "
```



```
INT16 -> dtype: int16 | min: 0 | max: 32767 | nbytes: 1685400
INT32 -> dtype: int32 | min: 0 | max: 2147483647 | nbytes: 3370800
```



	Secara visual, tampilannya sama sekali tidak berbeda (keduanya menghasilkan kanvas hitam). Namun, ukuran memorinya (nbytes) berbeda. Tipe data int32 menggunakan 4 byte per nilai, sedangkan int16 hanya 2 byte, sehingga int32 memakan memori dua kali lipat lebih besar dan memiliki jangkauan nilai maksimum/minimum yang jauh lebih luas.
3	Apa kegunaan baris <code>from google.colab.patches import cv2_imshow</code>, dan mengapa <code>cv2.imshow()</code> bawaan OpenCV tidak dapat dipakai di Google Colab? <pre>import cv2 as cv import numpy as np import urllib.request from google.colab.patches import cv2_imshow url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg' req = urllib.request.urlopen(url) arr = np.asarray(bytearray(req.read()), dtype=np.uint8) img = cv.imdecode(arr, cv.IMREAD_COLOR) # cv.imshow('GUI', img) # <--- Baris ini akan menghasilkan ERROR jika dijalankan di Colab cv2_imshow(img) # <--- Ini berfungsi menampilkan gambar di output cell</pre> Fungsi <code>cv2.imshow()</code> bawaan OpenCV membutuhkan antarmuka jendela GUI (Graphical User Interface) desktop dari sistem operasi untuk menampilkan gambar. Karena Google Colab berjalan di peramban web (browser) berbasis cloud, GUI desktop tersebut tidak tersedia, sehingga jika digunakan akan menyebabkan error. Modul <code>google.colab.patches</code> menyediakan fungsi <code>cv2_imshow</code> sebagai penambal (patch) agar citra dapat dirender sebagai gambar statis langsung di dalam sel output web.
4	Citra yang dibaca dengan <code>skimage.io.imread()</code> ditampilkan berdampingan dengan hasil <code>cv2.cvtColor(img, cv2.COLOR_BGR2RGB)</code> atas citra yang sama. Manakah yang menampilkan warna sebenarnya, dan manakah yang kanalnya tertukar? Buktikan dengan nilai piksel, bukan dengan pengamatan visual saja.

	<pre> from skimage import io import cv2 as cv import matplotlib.pyplot as plt url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg' img_asli_sk = io.imread(url) # Sudah RGB (Benar) img_salah_konversi = cv.cvtColor(img_asli_sk, cv.COLOR_BGR2RGB) # Dipaksa tukar (Salah) fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5)) ax1.imshow(img_asli_sk) ax1.set_title("skimage asli (Warna Benar / RGB)") ax2.imshow(img_salah_konversi) ax2.set_title("Setelah cvtColor (Warna Salah / Tertukar)") plt.show() print("Piksel [0,0] Skimage asli (RGB) :", img_asli_sk[0,0]) print("Piksel [0,0] Setelah cvtColor :", img_salah_konversi[0,0]) </pre> skimage asli (Warna Benar / RGB)  Piksel [0,0] Skimage asli (RGB) : [225 138 128] Setelah cvtColor (Warna Salah / Tertukar)  Piksel [0,0] Setelah cvtColor : [128 138 225] Citra yang dibaca dengan <code>skimage.io.imread()</code> menampilkan warna sebenarnya karena secara bawaan fungsi ini membaca berkas dengan urutan kanal RGB. Jika fungsi <code>cv.cvtColor(img, cv.COLOR_BGR2RGB)</code> dipaksakan pada citra tersebut, program justru akan membalik posisi kanal Red dan Blue, sehingga warnanya akan menjadi tertukar (salah). 5 Setelah citra ditampilkan dengan <code>figsize</code> yang jauh lebih besar, apakah nilai <code>img.shape</code> ikut berubah? Jelaskan perbedaan antara mengubah ukuran citra dan mengubah ukuran tampilan
--	---

```
import cv2 as cv
import numpy as np
import urllib.request
import matplotlib.pyplot as plt

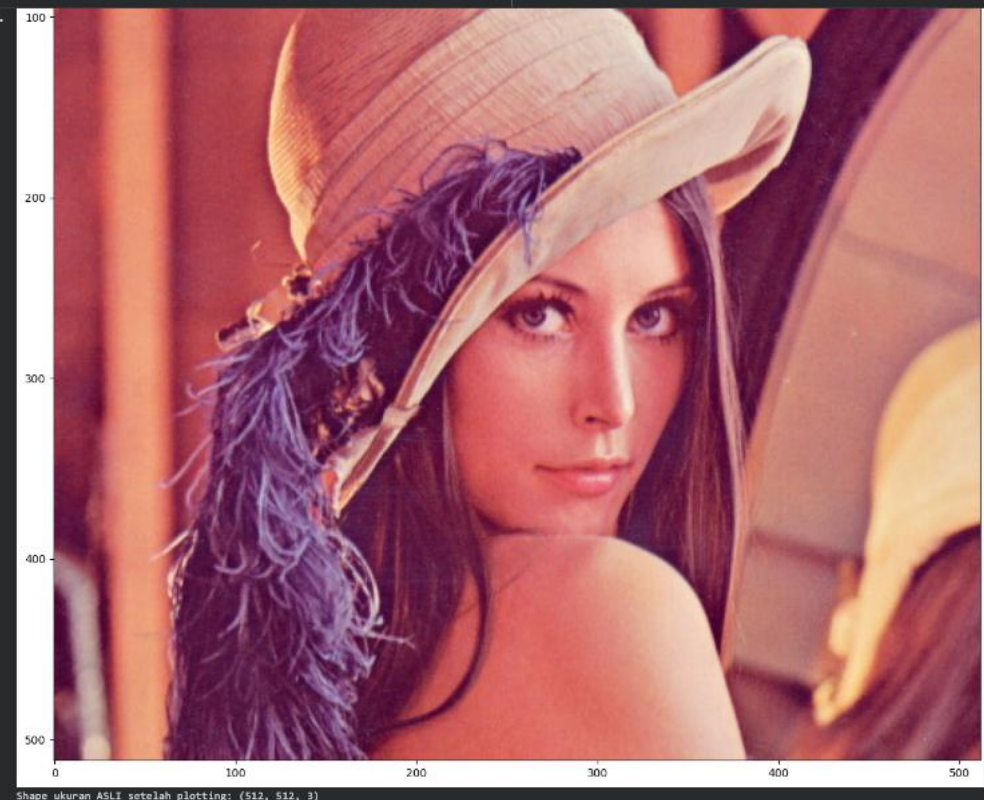
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg'
req = urllib.request.urlopen(url)
arr = np.asarray(bytearray(req.read()), dtype=np.uint8)
img = cv.imdecode(arr, cv.IMREAD_COLOR)

img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)

print("Shape ukuran ASLI sebelum plotting:", img.shape)

# Mengubah ukuran tampilan kanvas plot (figsize)
fig = plt.figure(figsize=(15, 15)) # Tampilan akan menjadi sangat besar
ax = fig.add_subplot(1, 1, 1)
ax.imshow(img_rgb)
ax.set_title("Tampilan dengan figsize (15,15)")
plt.show()

print("Shape ukuran ASLI setelah plotting:", img.shape)
```



Shape ukuran ASLI setelah plotting: (512, 512, 3)

Tidak, nilai `img.shape` sama sekali tidak berubah. Parameter `figsize` pada Matplotlib hanya memperbesar dimensi fisik kanvas gambar tempat plot dilukis di layar (dalam satuan inci). Hal ini murni urusan tampilan luarnya saja. Untuk benar-benar mengubah resolusi atau jumlah piksel citra secara permanen (sehingga parameter `shape` ikut berubah), kita wajib menggunakan `cv2.resize()` yang akan menghitung ulang susunan piksel aslinya.



TUGAS PRAKTIKUM – Penerapan

- 1 Tampilkan citra hanya pada kombinasi kanal Merah-Biru dan kanal Hijau-Biru saja, dengan kanal yang tidak dipakai dinolkan. Susun keduanya berdampingan dalam satu figure yang diberi judul.

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from skimage import io

# Membaca gambar dari URL (otomatis dalam format RGB)
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg'
img_rgb = io.imread(url)

# 1. Kombinasi Merah-Biru (Kanal Hijau dinolkan)
img_merah_biru = img_rgb.copy()
img_merah_biru[:, :, 1] = 0 # Indeks 1 adalah Green

# 2. Kombinasi Hijau-Biru (Kanal Merah dinolkan)
img_hijau_biru = img_rgb.copy()
img_hijau_biru[:, :, 0] = 0 # Indeks 0 adalah Red

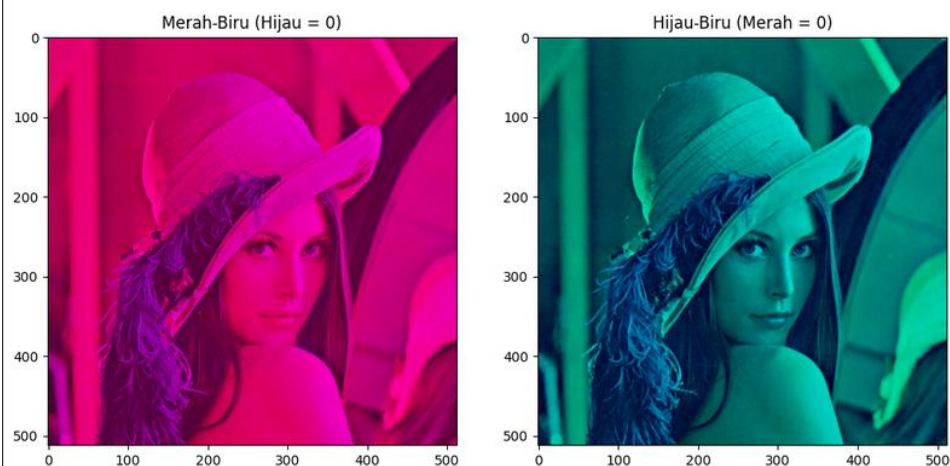
# Menampilkan berdampingan dalam satu figure
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))
fig.suptitle('Penerapan Manipulasi Kanal Warna', fontsize=16)

ax1.imshow(img_merah_biru)
ax1.set_title('Merah-Biru (Hijau = 0)')

ax2.imshow(img_hijau_biru)
ax2.set_title('Hijau-Biru (Merah = 0)')

plt.show()
```

Penerapan Manipulasi Kanal Warna



- 2 Tampilkan citra dalam tiga bentuk pencerminan: vertikal, horizontal, dan keduanya, dalam satu figure berisi tiga subplot yang masing-masing diberi label.

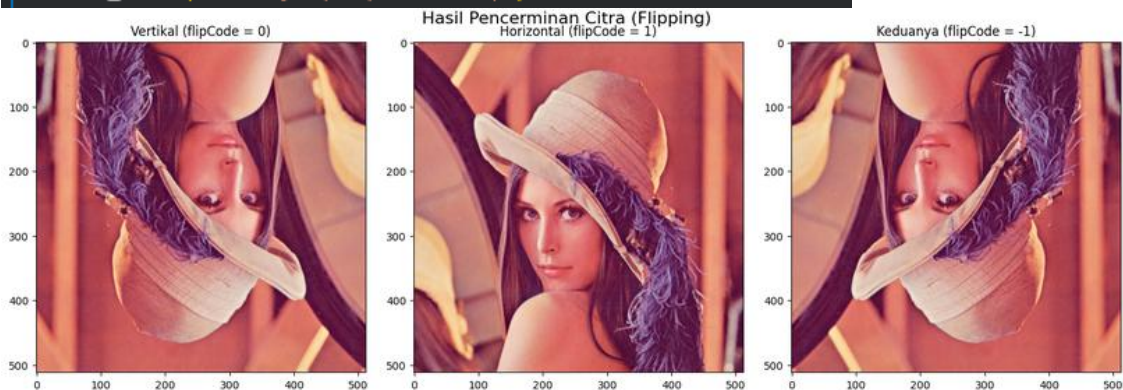
```
# Menggunakan citra yang sama dari variabel img_rgb di atas
# Melakukan flipping sesuai dokumentasi OpenCV
flip_vertikal = cv.flip(img_rgb, 0)
flip_horizontal = cv.flip(img_rgb, 1)
flip_keduanya = cv.flip(img_rgb, -1)

# Menampilkan dalam satu figure berisi 3 subplot
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Hasil Pencerminan Citra (Flipping)', fontsize=16)

ax1.imshow(flip_vertikal)
ax1.set_title('Vertikal (flipCode = 0)')

ax2.imshow(flip_horizontal)
ax2.set_title('Horizontal (flipCode = 1)')

ax3.imshow(flip_keduanya)
ax3.set_title('Keduanya (flipCode = -1)')
```



- 3 Gambarkan sebuah rectangle dan sebuah circle pada bagian wajah / badan dari foto aktivitas Anda sendiri (bukan pasfoto), lalu tambahkan teks berisi nama Anda dengan pilihan font, ukuran, dan warna yang Anda tentukan sendiri.

```
import cv2 as cv
import matplotlib.pyplot as plt
from google.colab import files

# =====
# TUGAS: Rectangle, Circle, dan Teks pada foto aktivitas
# =====

# 1. Menggunakan file upload langsung dari komputer
print("Silakan upload file foto Anda (misal: aktivitas.jpeg)")
uploaded = files.upload()

# Mengambil nama file yang baru saja diupload secara otomatis
file_path = next(iter(uploaded))

... Silakan upload file foto Anda (misal: aktivitas.jpeg)
      aktivitas.jpeg
      aktivitas.jpeg(image/jpeg) - 1050365 bytes, last modified: n/a - 100% done
      Saving aktivitas.jpeg to aktivitas (3).jpeg
```




```
# Membaca citra dan konversi BGR ke RGB
img_foto = cv.imread(file_path)
img_foto_rgb = cv.cvtColor(img_foto, cv.COLOR_BGR2RGB)

tinggi_f, lebar_f, _ = img_foto_rgb.shape

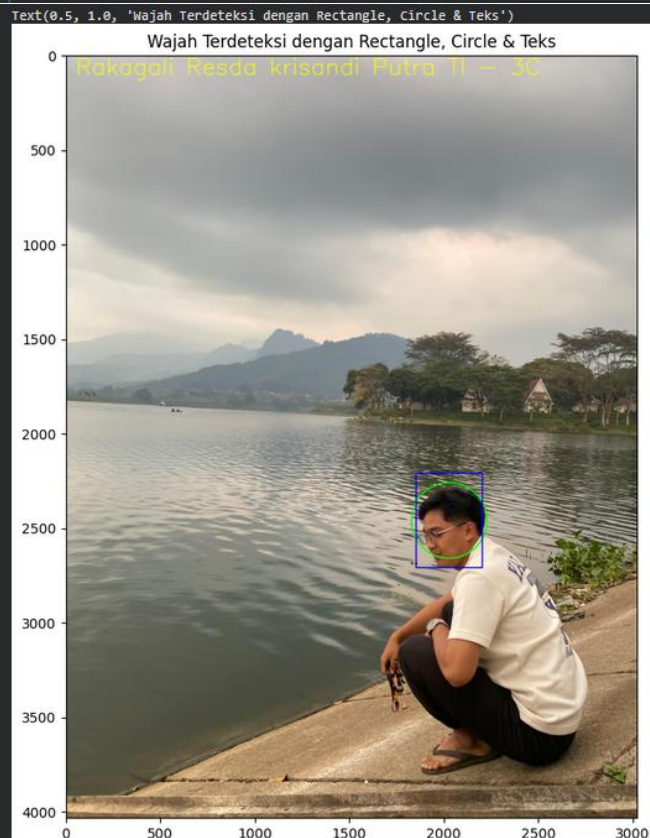
# Koordinat khusus yang disesuaikan dengan posisi wajah pada foto
x_wajah = int(lebar_f * 0.67) # Sekitar 67% dari sisi kiri
y_wajah = int(tinggi_f * 0.61) # Sekitar 61% dari atas
radius_lingkaran = 200
lebar_kotak = 175
tinggi_kotak = 250

# 2. Membuat Lingkaran (Circle) pada wajah (Warna Hijau = R:0, G:255, B:0)
cv.circle(img_foto_rgb, center=(x_wajah, y_wajah), radius=radius_lingkaran, color=(0, 255, 0), thickness=5)

# 3. Membuat Persegi Panjang (Rectangle) membingkai wajah (Warna Biru = R:0, G:0, B:255)
cv.rectangle(img_foto_rgb,
             pt1=(x_wajah - lebar_kotak, y_wajah - tinggi_kotak),
             pt2=(x_wajah + lebar_kotak, y_wajah + tinggi_kotak),
             color=(0, 0, 255), thickness=5)

# 4. Menambahkan teks berisi nama
font = cv.FONT_HERSHEY_SIMPLEX
cv.putText(img_foto_rgb, text='Rakagali Resda krisandi Putra TI - 3C', org=(50, 100), fontFace=font, fontScale=4.0,
           color=(255, 255, 0), thickness=4, lineType=cv.LINE_AA)

# Tampilkan Hasil
plt.figure(figsize=(10, 10))
plt.imshow(img_foto_rgb)
plt.title("Wajah Terdeteksi dengan Rectangle, Circle & Teks")
```





TUGAS PRAKTIKUM - Analisa

1	Rumuskan satu pertanyaan tentang citra digital yang dapat Anda jawab menggunakan kode dari modul ini. Contoh pertanyaan: berapa persen piksel pada foto KTM Anda yang lebih gelap dari nilai 100? apakah mengecilkan lalu membesarkan citra mengubah rata-rata intensitasnya? Tuliskan pertanyaan Anda, rancangan program, data hasilnya, dan kesimpulan Anda.
	1. Pertanyaan: Di antara ketiga kanal warna (Merah, Hijau, Biru) pada citra foto saya, kanal warna apakah yang paling mendominasi (memiliki rata-rata intensitas piksel tertinggi)? 2. Rancangan Program: Logika program untuk menjawab pertanyaan tersebut adalah: 1. Membaca citra dan mengonversinya dari BGR ke RGB agar urutannya benar (Red, Green, Blue). 2. Memisahkan citra menjadi matriks tunggal untuk setiap warna menggunakan fitur pengirisan (<i>slicing</i>) pada NumPy <code>img[:, :, kanal]</code>. 3. Menghitung nilai rata-rata (<code>np.mean</code>) dari seluruh nilai piksel (0-255) pada masing-masing matriks kanal warna. 4. Mencetak data hasilnya dan membandingkan angka rata-rata untuk menarik kesimpulan. Kode Program: <pre>import cv2 as cv import numpy as np from google.colab import files print("Silakan upload foto yang ingin dianalisa:") uploaded = files.upload() file_path = next(iter(uploaded)) # 1. Membaca gambar dan mengubah format ke RGB img = cv.imread(file_path) img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB) # 2. Mengambil (slicing) masing-masing kanal warna # Indeks 0 = Red, 1 = Green, 2 = Blue kanal_merah = img_rgb[:, :, 0] kanal_hijau = img_rgb[:, :, 1] kanal_biru = img_rgb[:, :, 2] # 3. Menghitung rata-rata intensitas dari tiap matriks kanal rata_merah = np.mean(kanal_merah) rata_hijau = np.mean(kanal_hijau) rata_biru = np.mean(kanal_biru) # 4. Menampilkan Data Hasil print("\n--- DATA HASIL ANALISA ---") print(f"Rata-rata Intensitas Merah (Red) : {rata_merah:.2f}") print(f"Rata-rata Intensitas Hijau (Green): {rata_hijau:.2f}") print(f"Rata-rata Intensitas Biru (Blue) : {rata_biru:.2f}")</pre>



```
# 5. Membuat Kesimpulan Otomatis
nilai_maksimal = max(rata_merah, rata_hijau, rata_biru)

print("\n--- KESIMPULAN ---")
if nilai_maksimal == rata_merah:
    print("Kanal MERAH (Red) memiliki nilai intensitas rata-rata tertinggi.")
    print("Artinya, secara keseluruhan, warna merah / nuansa hangat (warm) paling mendominasi foto ini.")
elif nilai_maksimal == rata_hijau:
    print("Kanal HIJAU (Green) memiliki nilai intensitas rata-rata tertinggi.")
    print("Artinya, secara keseluruhan, warna hijau paling mendominasi foto ini.")
else:
    print("Kanal BIRU (Blue) memiliki nilai intensitas rata-rata tertinggi.")
    print("Artinya, secara keseluruhan, warna biru / nuansa dingin (cool) paling mendominasi foto ini.")
```

3. Data Hasil:

Silakan upload foto yang ingin dianalisa:

aktivitas.jpeg

aktivitas.jpeg(image/jpeg) - 1050365 bytes, last modified: n/a - 100% done

Saving aktivitas.jpeg to aktivitas (4).jpeg

--- DATA HASIL ANALISA ---

Rata-rata Intensitas Merah (Red) : 150.61

Rata-rata Intensitas Hijau (Green): 144.44

Rata-rata Intensitas Biru (Blue) : 131.44

--- KESIMPULAN ---

Kanal MERAH (Red) memiliki nilai intensitas rata-rata tertinggi.

Artinya, secara keseluruhan, warna merah / nuansa hangat (warm) paling mendominasi foto ini.

4. Kesimpulan:

Berdasarkan data perhitungan dari array NumPy, dapat disimpulkan bahwa Kanal Merah memiliki nilai rata-rata tertinggi (150.61). Hal ini membuktikan bahwa citra foto yang saya gunakan didominasi oleh unsur warna merah, yang sangat wajar karena foto tersebut menonjolkan warna kulit manusia (*skin tone*) dan pencahayaan lampu ruangan yang cenderung hangat (*warm*).



TUGAS PRAKTIKUM - Aplikasi

- 1 Studi Kasus 1: Panitia sebuah kegiatan perlu membubuhkan label identitas pada ratusan foto dokumentasi. Foto berasal dari beberapa kamera dan ponsel, sehingga ukurannya bermacam-macam. Label harus terlihat seimbang pada semua foto tanpa disetel satu per satu. Fungsi yang sama dikenakan pada dua citra berbeda ukuran. Tinggi bar dan ukuran huruf ikut menyesuaikan, sehingga proporsinya tetap sama.

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from skimage import io

# 1. PERSIAPAN DATA UJI (Murni Slicing & Resize)
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg'
img_asli = io.imread(url)

# Memotong baris/kolom lalu mengubah ukuran agar sesuai rasio soal
img_landscape = cv.resize(img_asli[50:400, :], (900, 600))
img_portrait = cv.resize(img_asli[:, 100:400], (480, 760))

# 2. PROSES GAMBAR 1 (LANSKAP - 900 x 600)
hasil_land = img_landscape.copy()

# Ekstrak ukuran adaptif
h1, w1 = hasil_land.shape[:2]
bar_height1 = h1 // 12

# Perhitungan adaptif skala teks & ketebalan
skala_teks1 = min(w1 / 600.0, bar_height1 / 35.0)
tebal_teks1 = max(1, int(skala_teks1 * 2.5))

# Penggambaran kotak
cv.rectangle(hasil_land, pt1=(0, h1 - bar_height1), pt2=(w1, h1), color=(15, 15, 15), thickness=-1)

# Penggambaran teks
margin_x1 = int(w1 * 0.02)
margin_y1 = int(h1 - (bar_height1 * 0.3))
cv.putText(hasil_land, text="Dokumentasi PCVK - 2026", org=(margin_x1, margin_y1),
           fontFace=cv.FONT_HERSHEY_SIMPLEX, fontScale=skala_teks1,
           color=(255, 255, 255), thickness=tebal_teks1, lineType=cv.LINE_AA)

# 3. PROSES GAMBAR 2 (POTRET - 480 x 760)
hasil_port = img_portrait.copy()

# Ekstrak ukuran adaptif
h2, w2 = hasil_port.shape[:2]
bar_height2 = h2 // 12

# Perhitungan adaptif skala teks & ketebalan
skala_teks2 = min(w2 / 600.0, bar_height2 / 35.0)
tebal_teks2 = max(1, int(skala_teks2 * 2.5))

# Penggambaran kotak
cv.rectangle(hasil_port, pt1=(0, h2 - bar_height2), pt2=(w2, h2), color=(15, 15, 15), thickness=-1)

# Penggambaran teks
margin_x2 = int(w2 * 0.02)
margin_y2 = int(h2 - (bar_height2 * 0.3))
cv.putText(hasil_port, text="Dokumentasi PCVK - 2026", org=(margin_x2, margin_y2),
           fontFace=cv.FONT_HERSHEY_SIMPLEX, fontScale=skala_teks2,
           color=(255, 255, 255), thickness=tebal_teks2, lineType=cv.LINE_AA)
```

	<pre># 4. PENAMPILAN HASIL fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 7)) ax1.imshow(hasil_land) ax1.set_title(f"Masukan 900 x 600 px\nKeluaran: bar = 600//12 = {bar_height1} px") ax1.axis('off') ax2.imshow(hasil_port) ax2.set_title(f"Masukan 480 x 760 px\nKeluaran: bar = 760//12 = {bar_height2} px") ax2.axis('off') plt.tight_layout() plt.show()</pre> Masukan 900 x 600 px Keluaran: bar = 600//12 = 50 px  Dokumentasi PCVK – 2026 Masukan 480 x 760 px Keluaran: bar = 760//12 = 63 px  Dokumentasi PCVK – 2026
2	Studi Kasus 2: Marketplace seperti Tokopedia dan Shopee menampilkan foto produk dalam bingkai persegi 1:1. Foto penjual yang berukuran potrait atau lanskap akan terpotong atau terlihat tidak proporsional bila diunggah begitu saja. Bangun program yang mengubah foto berukuran apa pun menjadi thumbnail persegi tanpa mengubah rasio aslinya, dengan sisa ruang diisi kanvas berwarna. Dalam hal ini, rasio asli tetap terjaga, citra terpasang tepat di tengah kanvas, tambahkan logo nama toko berukuran proporsional pada gambar.



```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from skimage import io

# 1. PERSIAPAN DATA UJI (Murni Slicing & Resize)
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg'
img_asli = io.imread(url)

# Membuat dua versi citra dengan ukuran (resolusi) yang berbeda (Lanskap dan Potret)
img_landscape = cv.resize(img_asli[50:400, :], (800, 450))
img_portrait = cv.resize(img_asli[:, 100:400], (500, 750))

# 2. PROSES PRODUK 1 (LANSKAP) -> THUMBNAIL 1:1

h1, w1 = img_landscape.shape[:2]

# Menentukan ukuran persegi (sisi terpanjang dari gambar)
sisi1 = max(h1, w1)

# Membuat kanvas putih persegi 1:1 sesuai ukuran sisi terpanjang
# Menggunakan np.full
kanvas_land = np.full((sisi1, sisi1, 3), 255, dtype=np.uint8)
# Menghitung titik tengah (offset) untuk menempelkan gambar
y_awal1 = (sisi1 - h1) // 2
x_awal1 = (sisi1 - w1) // 2

# Menempelkan gambar ke kanvas menggunakan Slicing NumPy
kanvas_land[y_awal1:y_awal1+h1, x_awal1:x_awal1+w1] = img_landscape

# Menambahkan border
tebal_bingkai1 = max(2, int(sisi1 * 0.02))
cv.rectangle(kanvas_land, pt1=(0, 0), pt2=(sisi1, sisi1), color=(150, 150, 150), thickness=tebal_bingkai1)

# Menambahkan Teks Logo Toko proporsional di sudut kiri atas
skala_logo1 = sisi1 / 600.0
tebal_logo1 = max(1, int(skala_logo1 * 2))
cv.putText(kanvas_land, text="TOKO PCVK", org=(int(sisi1 * 0.05), int(sisi1 * 0.1)),
           fontFace=cv.FONT_HERSHEY_DUPLEX, fontScale=skala_logo1,
           color=(0, 0, 255), thickness=tebal_logo1, lineType=cv.LINE_AA)

# 3. PROSES PRODUK 2 (POTRET) -> THUMBNAIL 1:1

h2, w2 = img_portrait.shape[:2]

# Menentukan ukuran persegi (sisi terpanjang dari gambar)
sisi2 = max(h2, w2)

# Membuat kanvas putih persegi 1:1
kanvas_port = np.full((sisi2, sisi2, 3), 255, dtype=np.uint8)

# Menghitung titik tengah (offset)
y_awal2 = (sisi2 - h2) // 2
x_awal2 = (sisi2 - w2) // 2

# Menempelkan gambar ke kanvas menggunakan Slicing NumPy
kanvas_port[y_awal2:y_awal2+h2, x_awal2:x_awal2+w2] = img_portrait

# Menambahkan border (bingkai abu-abu)
tebal_bingkai2 = max(2, int(sisi2 * 0.02))
cv.rectangle(kanvas_port, pt1=(0, 0), pt2=(sisi2, sisi2), color=(150, 150, 150), thickness=tebal_bingkai2)

# Menambahkan Teks Logo Toko proporsional di sudut kiri atas
skala_logo2 = sisi2 / 600.0
tebal_logo2 = max(1, int(skala_logo2 * 2))
cv.putText(kanvas_port, text="TOKO PCVK", org=(int(sisi2 * 0.05), int(sisi2 * 0.1)),
           fontFace=cv.FONT_HERSHEY_DUPLEX, fontScale=skala_logo2,
           color=(0, 0, 255), thickness=tebal_logo2, lineType=cv.LINE_AA)
```

4. PENAMPILAN HASIL

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5))
fig.suptitle('Hasil Thumbnail 1:1 untuk Marketplace', fontsize=16)

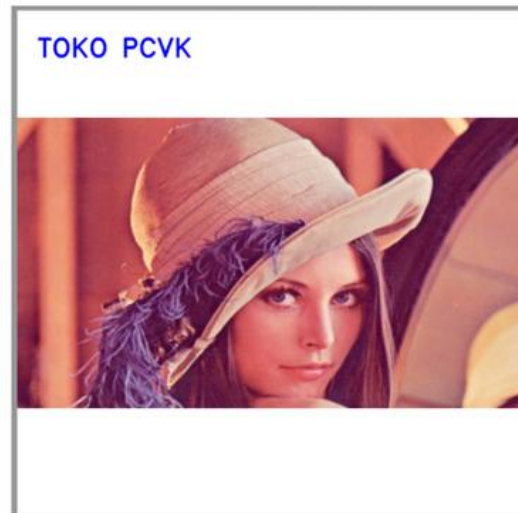
ax1.imshow(kanvas_land)
ax1.set_title(f'Masukan {w1}x{h1} -> Thumbnail {sisi1}x{sisi1}')
ax1.axis('off')

ax2.imshow(kanvas_port)
ax2.set_title(f'Masukan {w2}x{h2} -> Thumbnail {sisi2}x{sisi2}')
ax2.axis('off')

plt.tight_layout()
plt.show()
```

Hasil Thumbnail 1:1 untuk Marketplace

Masukan 800x450 -> Thumbnail 800x800



Masukan 500x750 -> Thumbnail 750x750

